How fast is `search` on a k-d tree? Like for regular *BST*s, its running time is proportional to the height of the tree; if we keep the tree balanced, therefore, the running time will be `O(log(n))` for a tree holding `n` points.

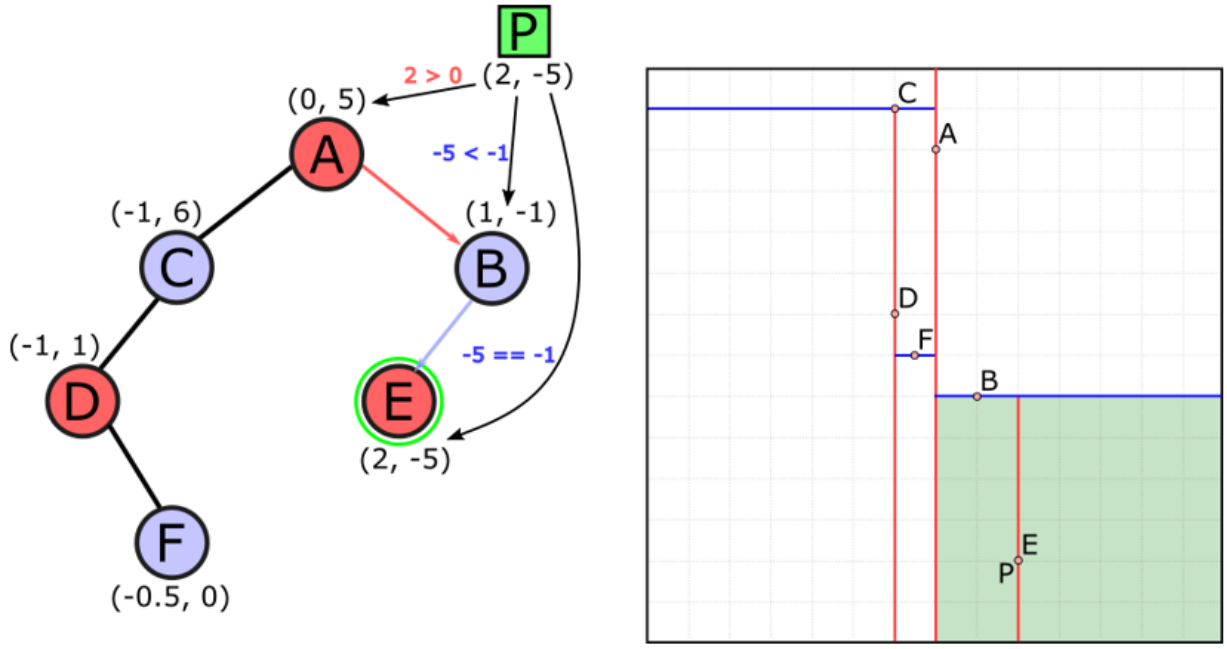


Figure 9.10 An example of a successful search on a k-d tree (2-D). Here points `P` and `E` are coincident, and the highlighted region corresponds to the subtree rooted at node `E`.

9.3.2 Insert

As we have seen on *BST*s, insertion can be implemented in two steps; the first step is running a search for the point to add, which will either find the point is already in the tree, or stop at its parent-to-be, the node to which the new point should be added as a child. If the point is already in the tree, then what we do next depends on the policy we adopt on duplicates: if duplicates are not allowed, we might ignore the